

Module 4: Web Application Development Concepts

Jayvee Christopher N. Vibar, DIT

Web Application Development Concepts

- Requirements Analysis for Web Information Systems
- System Modeling
- Frameworks, Security, and Prototyping

Learning Objectives

- Understand requirements analysis for web systems
- Learn common system modeling techniques
- Understand web application frameworks
- Learn authentication and authorization concepts
- Explore rapid prototyping and low-code platforms

What is a Web Information System (Web IS)?

- Information system delivered through web technologies
- Accessible through browsers
- Integrates databases, servers, and user interfaces
- Supports organizational processes

Examples of Web Information Systems

- University information systems
- Online banking
- E-commerce platforms
- Government service portals

Importance of Requirements Analysis

- Defines what the system should do
- Identifies user needs
- Prevents costly redesign
- Ensures system success

Types of Requirements

- Functional requirements
- Non-functional requirements
- User requirements
- System requirements

Functional Requirements

- Describe system behavior
- Define specific operations
- Examples: login, search, register, submit forms

Non-Functional Requirements

- Performance
- Security
- Usability
- Scalability
- Reliability

Stakeholder Identification

- End users
- Administrators
- Developers
- Managers

Requirements Gathering Techniques

- Interviews
- Surveys and questionnaires
- Observation
- Document analysis
- Workshops

User Stories

- Short description of functionality
- Written from user perspective
- Common in Agile development
 - E.g. 'As a student, I want to view my grades online.'

Requirements Documentation

- Software Requirements Specification (SRS)
- User stories
- Use case descriptions
- Prototypes

Software Requirements Specification (SRS)

Project: University Enrollment System (UES)

Version: 1.0

Date: March 13, 2026

1. Introduction

Purpose: To define the functional and non-functional requirements for the UES to automate course registration.

Scope: The system will manage student records, course catalogs, and enrollment processing, interfacing with the central SIS and Payment Gateway.

2. Overall Description

System Context:

User Classes: * **Student:** Enrolls in courses.

Registrar: Manages subjects and schedules.

3. Functional Requirements

These define the specific "shall" statements for the system:

REQ-01 (Course Selection): The system shall allow students to browse available subjects filtered by Program and Year Level.

REQ-02 (Conflict Detection): The system shall validate for schedule overlaps and display an error if a conflict is detected.

REQ-03 (Prerequisite Check): The system shall block enrollment if the student has not completed the required prerequisite subjects.

REQ-04 (Fee Assessment): The system shall calculate the total tuition and laboratory fees based on selected units.

4. Non-Functional Requirements

Performance: The system shall process enrollment requests within 3 seconds under a load of 5,000 concurrent users.

Security: All user data transmissions shall be encrypted via HTTPS/TLS 1.3.

Reliability: The system shall maintain 99.9% uptime during the defined enrollment period.

Usability: The interface shall adhere to WCAG 2.1 accessibility standards.

5. Use Case Summary

UC-01: Enroll in Subject

UC-02: Manage Course Catalog (Registrar)

UC-03: Process Payment (External/System)

6. Data Requirements (Database Schema)

The system shall utilize a relational database with the following primary entities:

Student, Program, Subject, Section, Enrollment.

Introduction to System Modeling

- Visual representation of systems
- Helps understand complex processes
- Used during system analysis and design

Common Modeling Techniques

- Use Case Diagrams
- Data Flow Diagrams (DFD)
- Entity Relationship Diagrams (ERD)

Use Case Diagrams

- Show interactions between users and system
- Identify actors and system functions
- Focus on system behavior

Actors in Use Case Diagrams

- External users interacting with the system
- Examples: student, admin, cashier

Example Use Cases

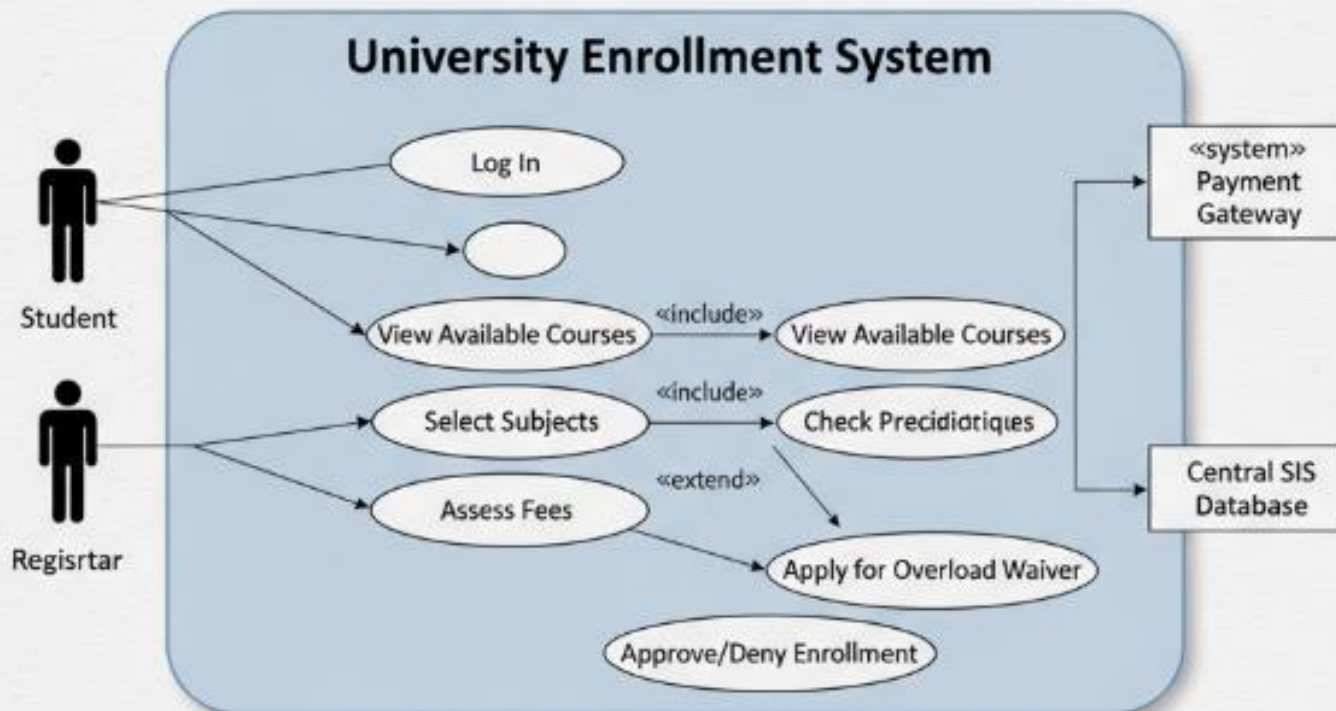
- Login
- Register account
- Submit application
- Generate report

Benefits of Use Case Modeling

- Clarifies system scope
- Improves communication
- Helps identify system functionality

e.g. How actor interacts with the system.





Data Flow Diagrams (DFD)

- Shows how data moves in a system
- Identifies processes and data stores
- Represents inputs and outputs

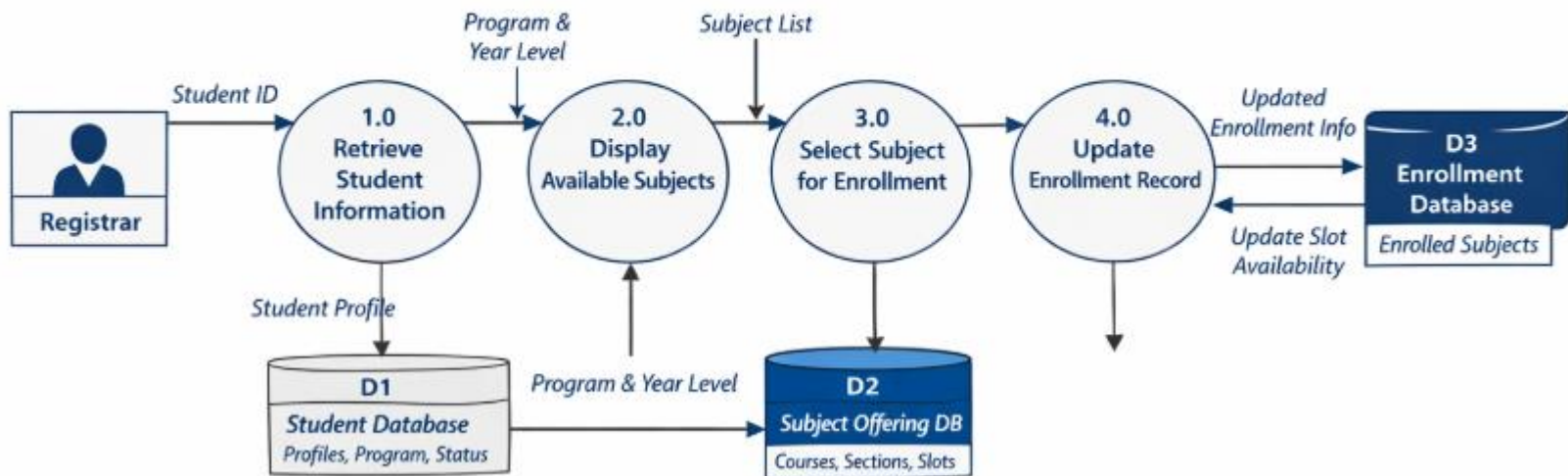
DFD Components

- Process
- Data store
- External entity
- Data flow

DFD Levels

- Context diagram (Level 0)
- Level 1 DFD
- Level 2 or deeper decomposition

Registrar Selecting a Subject for Student Enrollment



Benefits of DFD

- Shows system processes clearly
 - This can be broad or specific e.g. use leveling
- Helps identify data sources
 - From and to
- Useful for process analysis

Entity Relationship Diagram (ERD)

- Models database structure
- Defines entities and relationships
- Used for database design

Entities in ERD

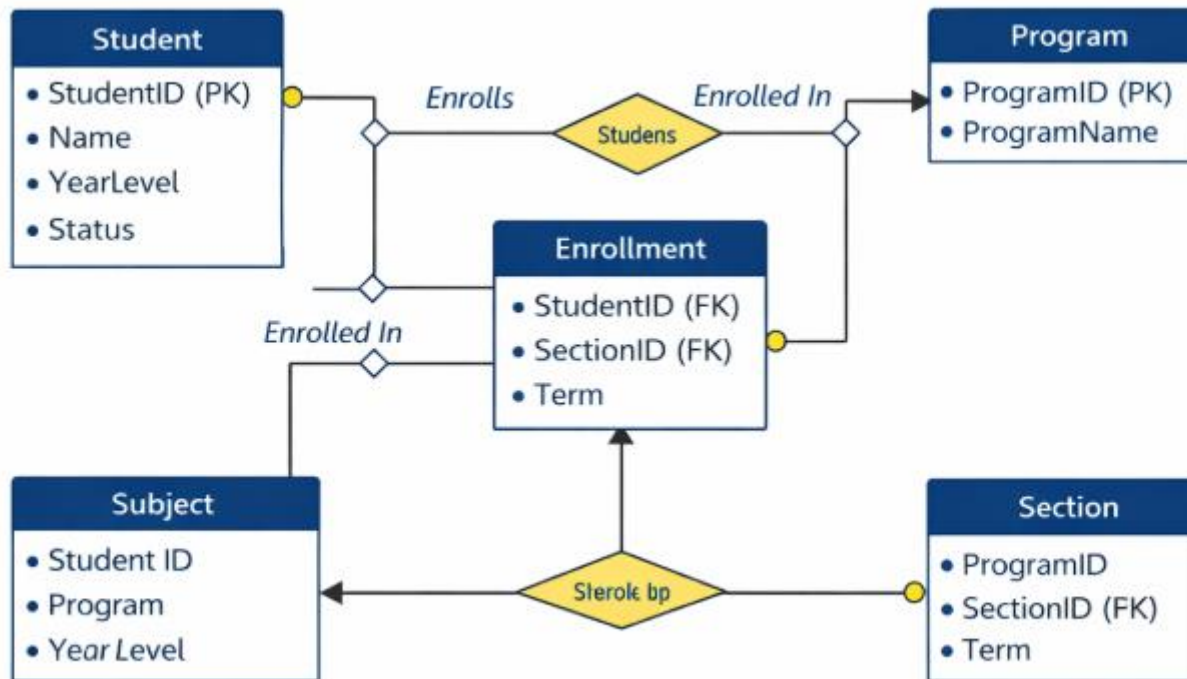
- Represent objects or concepts
- Examples: Student, Course, Instructor

Relationships in ERD

- Connections between entities
- One-to-one
- One-to-many
- Many-to-many

Attributes in ERD

- Properties of entities
- Examples: StudentID, Name, Email



Later, we can generate the physical schema based on our ERD

Sample MySQL Schema for Student Enrollment System

```
CREATE TABLE Student( {  
    StudentID INT,  
    Name VARCHAR(100),  
    YearLevel INT,  
    Status INT VARCHAR(50),  
};
```

```
CREATE TABLE Program( {  
    ProgramID INT,  
    ProgramName VARCHAR(100);  
};
```

```
CREATE TABLE Subject( {  
    SubjectID INT,  
    ProgramID INT,  
    YearLevel INT,  
};
```

```
CREATE TABLE Section( {  
    StudentID INT,  
    ProgramID INT,  
    Term VARCHAR(20) [10],  
};
```

```
CREATE TABLE Section ( {  
    SectionID INT,  
    ProgramID INT, 00),  
    Term VARCHAR(20), [20];  
};
```

```
CREATE TABLE Enrollment ( {  
    StudentID INT,  
    SectionID INT; 00 StudentID> ),  
    Term VARCHAR(20), 505.50TE }  
}; PRIMARY KEY StudentID*sectionID
```

Introduction to Web Application Frameworks

- Tools that simplify web development
- Provide reusable components
- Standardize application structure

Benefits of Frameworks

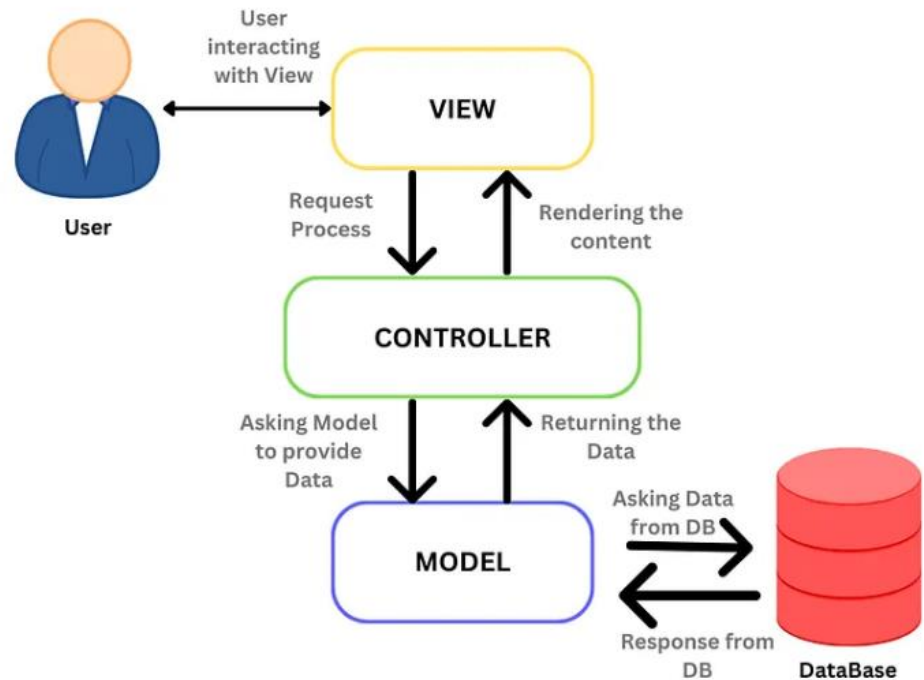
- Faster development
- Built-in security features
- Better maintainability
- Code organization

Examples of Web Frameworks

- Laravel / Codeigniter (PHP)
- Django (Python)
- Ruby on Rails
- Express.js (Node.js)

MVC Architecture

- Model
- View
- Controller



Model Component

- Handles business logic
- Manages database interactions

View Component

- User interface layer
- Displays data to users

Controller Component

- Handles user requests
- Connects model and view

Authentication Concepts

- Process of verifying identity
- Ensures user is who they claim to be

Common Authentication Methods

- Username and password
- Two-factor authentication
- Biometric authentication
- OAuth / social login (Open Authorization)

Authorization Concepts

- Determines user permissions
- Controls what users can access

Role-Based Access Control (RBAC)

- Users assigned roles
- Roles define permissions
- Common in enterprise systems

Session Management

- Maintains user state across requests
- Uses cookies or session IDs
- Essential for login systems

Security Best Practices

- Use HTTPS
- Hash passwords
- Validate input
- Prevent SQL injection

Rapid Prototyping

- Quickly build system models
- Validate requirements early
- Improve user feedback

Student Registration Form

Basic Information



Full Name

First Name

Middle Name

Last Name

Student Number

e.g. 11183021

Year Level

Please Select

▼

for Incoming A.Y. '12-'13

Degree Program

Please Select

▼

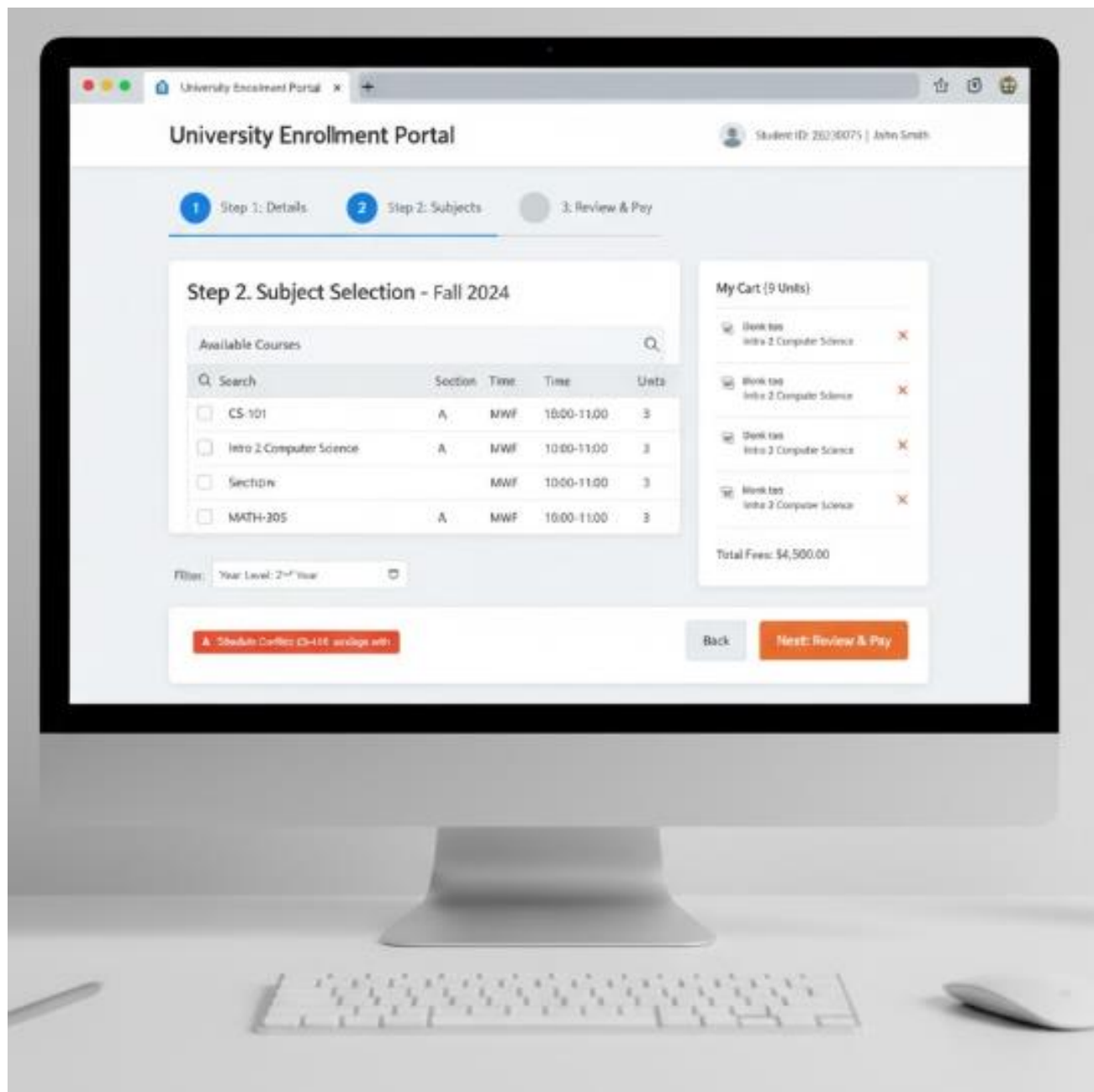
Email

Phone Number

Network Provider

Please Select

▼



Benefits of Rapid Prototyping

- Faster feedback
- Reduced development risk
- Better user satisfaction

Low-Code Platforms

- Development with minimal coding
- Use visual interfaces
- Rapid application development

No-Code Platforms

- Applications built without programming
- Drag-and-drop components
- Focus on business logic

Examples of Low-Code Tools

- OutSystems
- Microsoft Power Apps
- Bubble
- Zoho Creator

Top Low-Code Development Tools

Tool 1

appian

Tool 2

 **quixy**

Tool 3

 **mendix**

Tool 4

 **QUICK
BASE**

Tool 5

 **outsystems**

Tool 6

 **Lightning**

Tool 7

 **Zoho
Creator**

Tool 8

**Microsoft
PowerApps** 

Tool 9

 **nintex**

Advantages of Low-Code/No-Code

- Faster development
- Reduced technical barrier
- Lower development cost

Limitations of Low-Code Platforms

- Limited customization
- Vendor dependency
- Scalability concerns

Summary

- Requirements analysis defines system needs
- System modeling visualizes processes
- Frameworks accelerate development
- Security is critical
- Prototyping improves system design

Discussion Questions

- Why is requirements analysis critical?
- When should DFD be used instead of ERD?
- What are the advantages of web frameworks?
- When should low-code platforms be used?

Thank You.